

Пояснительная записка к проекту

«Веб-приложение для проведения интерактивных опросов»

Студентка: Лапенко Карина Александровна

Группа: М8О-315Б-23

Проект: MVP платформы интерактивных квизов для мероприятий

1. Цель и результат

Разработан MVP веб-приложения для интерактивных квизов в реальном времени: организатор создает и запускает комнату, участники подключаются по коду, отвечают только во время демонстрации вопроса, получают баллы и итоговый лидерборд.

2. Выбор средств разработки

- Frontend: Vue 3, Vite, Pinia, Vue Router, Socket.IO Client.
- Backend: Node.js, Express, Socket.IO, SQLite через better-sqlite3.
- Безопасность: JWT, bcryptjs, проверка ролей, ownership, CORS, защита правильных ответов.
- Поставка: Docker Compose, nginx для frontend/API/Socket.IO, SQLite volume.

3. Анализ конкурентов и выбранные настройки

Сервис	Что важно у конкурентов	Как учтено в MVP
Kahoot	shuffle, live settings, блокировка входа, очки за скорость и правильность	таймер, перемешивание, speed scoring, live leaderboard
Wayground / Quizizz	attempt limits, show answer, leaderboard, отчеты по попыткам	лимит попыток, запрет смены ответа, аналитика, экспорт
Mentimeter	quiz competition, scoring mode, results page, export	режимы scoring, итоговые результаты, CSV-выгрузка

4. Архитектура и модель данных

Сущности базы данных: users, quizzes, questions, question_options, quiz_launches, launch_participants, answers. Такая модель хранит структуру квиза отдельно от live-сессий, поэтому один квиз можно запускать много раз и сохранять результаты каждой комнаты.

REST API отвечает за авторизацию, квизы, историю, аналитику и экспорт. Socket.IO отвечает за live-комнаты: ведущий отправляет start/next/finish, участники получают ожидание, countdown и текущий вопрос, а ответы принимаются только пока вопрос активен на сервере.

5. Реализованная функциональность

- Регистрация и авторизация пользователей с ролями организатора и участника.
- Конструктор квизов: категории, таймер, баллы, одиночный/множественный выбор, текстовые и image-url вопросы, перемешивание вопросов/ответов, запрет позднего входа, запрет изменения ответа, попытки, режим начисления баллов.
- Live-прохождение: код комнаты, экран ожидания, обратный отсчет, синхронная выдача вопросов, счетчик ответивших и не успевших, подтверждение ответа, финальный лидерборд.
- Личный кабинет: история созданных и проведенных квизов у организатора, история участия у участника.
- Админ-аналитика: KPI сессии, график точности по вопросам, распределение вариантов ответа, прогресс участников, таблица результатов и CSV-экспорт.

6. Административная аналитика

Админ-режим сделан как рабочая панель ведущего: она показывает не только итоговые места, но и качество прохождения. Ведущий видит количество участников, среднюю точность, завершенность, средний темп, самый сложный вопрос, графики точности, распределение ответов, прогресс каждого участника и CSV-экспорт. Это закрывает требование статистики по проведенному квизу и делает приложение полезным для преподавателя или организатора мероприятия.

7. Тестирование

- backend: npm run smoke и npm run qa прошли успешно; проверены positive и negative сценарии API/WebSocket.
- frontend: npm run build и npm run e2e прошли успешно; Playwright проверяет полный путь организатора и участника в реальном браузере.
- security: проверены protected routes, чужой доступ к квизам, аналитике и экспорту, запрет чтения правильных ответов участником, CORS.
- dependencies: npm audit backend/frontend - found 0 vulnerabilities.
- docker: docker compose config проходит; фактическая сборка выполняется после запуска Docker Desktop командой docker compose up --build -d.

Дополнительно подготовлен протокол учебного пользовательского тестирования на 5 человек: docs/user-testing-report.md. Скриншоты e2e и дизайн-галерея сохранены в docs/qa-screenshots и docs/design-gallery.

8. Соответствие требованиям

Требование	Реализация
Авторизация	JWT + bcrypt, роли organizer/participant, защищенные маршруты
Создание квизов	Настройки, категории, вопросы, варианты, тип ответа, изображения

Realtime	Socket.IO комнаты, start/next/finish, countdown, answer progress
Баллы и победители	scoring modes, leaderboard, детализация по вопросам
Личный кабинет	История запусков и участия
Тестирование	Smoke, QA, Playwright e2e, audit, Docker config, user-testing report

9. Ссылки для загрузки результата

- Figma: <https://www.figma.com/design/GnJooVBRAqkXPla72dZiGL/vk-project?node-id=0-1&t=eb8g0nYV48JD1dkn-1>
- Miro-ready материалы: docs/miro-board.md (перенести на доску Miro и вставить публичную ссылку в docs/submission-links.md)
- Репозиторий: локально инициализирован; после публикации вставить GitHub/GitLab URL в docs/submission-links.md
- Локальный запуск: docker compose up --build -d или отдельный запуск backend/frontend по README.md

10. Итог

Проект готов как защищаемый MVP: приложение запускается, проходит автоматические проверки, поддерживает полный live-сценарий квиса, сохраняет данные и дает организатору понятную аналитику. Оставшиеся внешние действия перед загрузкой на платформу: опубликовать репозиторий, перенести Miro-ready материалы на реальную Miro-доску и вставить публичные ссылки.